Understanding Spring Boot Actuator

Wensen Ma · Follow

6 min read · Jan 21, 2024

Spring Boot Actuators

Spring Boot Actuator is a powerful feature in the Spring Boot framework, offering deep insights into the runtime aspects of Spring Boot applications. This tool is essential for monitoring, managing, and understanding application behavior without the need for external monitoring systems. This article will walk you through what Spring Boot Actuator is, how it works, and why it's beneficial.

What is Spring Boot Actuator? Spring Boot Actuator is a sub-project of Spring Boot. It adds several production-grade features to your application, allowing

you to monitor and interact with it when it's deployed in a production environment.

Key Features of Spring Boot Actuator

- Health Checks: Provides information about the application's health status.
- Metrics Collection: Gathers various metrics such as HTTP requests, CPU usage, memory usage, etc.
- Application Info: Displays general information about the application, such as Git properties, build information.
- Dynamic Configuration: Offers mechanisms to refresh configuration properties on the fly.

How Does Spring Boot Actuator Work?

Enabling Spring Boot Actuator:

- To use Actuator, you need to add the `spring-boot-starter-actuator` dependency to your Spring Boot project's build configuration file (either `pom.xml` for Maven or `build.gradle` for Gradle).

Exposing Actuator Endpoints:

- Once the dependency is included, Spring Boot Actuator exposes a number of REST endpoints by default.
- These endpoints are accessible via HTTP and provide information related to the application's health, metrics, environment properties, etc.
- Commonly used endpoints include `/actuator/health`, `/actuator/info`, `/actuator/metrics`, etc.

Configuring Actuator Endpoints:

- By default, not all endpoints are exposed due to security reasons.
- You can configure which endpoints to expose by modifying the `application.properties` or `application.yml` file.
- For example, to expose all endpoints, you can set `management.endpoints.web.exposure.include=*`.

Health Indicators:

- The `/actuator/health` endpoint provides basic health information of your application.
- You can extend or customize health indicators by implementing the `HealthIndicator` interface.

Custom Metrics:

- Spring Boot Actuator can be integrated with external monitoring systems like Prometheus.
- You can also create custom metrics that are specific to your application's needs.

Security Considerations:

- Actuator endpoints may expose sensitive information. It's crucial to secure these endpoints, especially in a production environment.
- This can be achieved through Spring Security by restricting access to authenticated users or specific roles.

Benefits of Using Spring Boot Actuator

- Easy Monitoring and Management: Actuator provides essential features for monitoring and managing your application, making it easier to maintain good health and performance.
- Minimal Configuration: It requires minimal configuration and can be quickly integrated into your Spring Boot application.
- Customizability: Offers customization options to cater to specific monitoring needs.

- Real-time Application Insights: Enables real-time insights into the application, which is vital for identifying and resolving issues promptly.

Spring Boot Actuator is an indispensable tool for developers and system administrators alike, offering a wealth of information and control over Spring Boot applications. Its integration into Spring Boot applications paves the way for effective monitoring, management, and understanding of the application's behavior in a production environment. By leveraging Actuator, teams can ensure their applications are running smoothly, securely, and efficiently.

Behind the Scenes of Spring Boot Actuator

1. Integration with Spring Boot:

- Automatic Configuration: When you include the `spring-boot-starter-actuator` dependency in your project, Spring Boot auto-configures the Actuator endpoints. This is in line with Spring Boot's philosophy of convention over configuration.

- Conditional on Class and Property Conditions: Spring Boot's auto-configuration classes use `@ConditionalOnClass` and `@ConditionalOnProperty` annotations to check if Actuator and its dependencies are on the classpath, and if certain properties are set before configuring Actuator.

2. Endpoint Exposure Mechanism:

- Actuator Endpoints as Spring Beans: Each Actuator endpoint is a Spring bean, which gets registered in the Spring application context. These beans are picked up and exposed based on the application's configuration.
- MVC and WebFlux Support: Depending on whether your application uses Spring MVC or Spring WebFlux, Actuator auto-configures the necessary web controllers or handlers to expose these endpoints over HTTP.

3. Gathering Application Data:

- Health Indicators: Actuator uses a variety of `HealthIndicator` beans to report the health of different parts of your application (like database, disk space, custom components). These beans gather data from various parts

of your application and report it back when the `/actuator/health` endpoint is queried.

- Metrics Collection: For metrics, Actuator integrates with Micrometer — an application metrics facade that supports numerous monitoring systems. Actuator aggregates and serves these metrics through the `/actuator/metrics` endpoint.

4. Dynamic Configuration and Refresh Scope:

- @RefreshScope: For dynamic configuration management, beans annotated with `@RefreshScope` can be re-initialized at runtime with new configurations fetched from the Config Server, without restarting the application. This works in tandem with the `/actuator/refresh` endpoint.
- Environment Changes: Actuator listens for changes in the application's environment and updates the configuration properties accordingly when the `/actuator/refresh` endpoint is triggered.

5. Security and Actuator:

- Filtering Sensitive Data: Actuator automatically filters out sensitive information from its responses unless explicitly configured to expose

this data.

- Secure Endpoints: In conjunction with Spring Security, Actuator secures its endpoints, ensuring that only authenticated users with the necessary roles can access sensitive data.

6. Customization and Extensibility:

- Custom Endpoints: You can create custom Actuator endpoints by implementing the `Endpoint` interface, giving you the flexibility to expose application-specific data and operations.
- Extending Existing Endpoints: Existing Actuator endpoints can also be extended to add additional data or to change the existing behavior to suit specific requirements.

Spring Boot Actuator operates seamlessly with Spring Boot, leveraging its auto-configuration capabilities and integrating deeply with the application context and security framework. It provides a powerful and flexible way to monitor and manage applications, with support for customization and extensibility. By understanding these internal workings, developers can better leverage Actuator's capabilities for robust application monitoring and management.

Optimal Utilization of Spring Boot Actuator in Professional Environments

The standard way most companies use Spring Boot Actuator revolves around a few key practices that maximize its benefits for application monitoring and management. While the specific implementation details might vary depending on the company's infrastructure and requirements, the general approach typically includes the following elements:

1. Basic Setup and Configuration

- Include Actuator Dependency: Companies start by adding the `spring-boot-starter-actuator` dependency to their Spring Boot applications.
- Expose Necessary Endpoints: They configure Actuator to expose various endpoints. Commonly exposed endpoints include `/health`, `/info`, `/metrics`, etc. Sensitive endpoints are often restricted or secured.

2. Health Checks and Readiness/Liveness Probes

- Health Endpoint: The `/health` endpoint is widely used for health checks. In containerized environments (like Kubernetes), it's often used for readiness and liveness probes.

- Custom Health Indicators: Companies frequently implement custom health indicators to check the status of specific parts of their application, like database connectivity, external service availability, etc.

3. Metrics Collection and Monitoring

- Integration with Monitoring Tools: Actuator's metrics are often integrated with external monitoring systems like Prometheus, which in turn can be used with visualization tools like Grafana. This setup provides a powerful monitoring solution.
- Custom Metrics: Beyond default metrics, companies often define custom metrics specific to their application's operational or business needs.

4. Application Info and Management

- Info Endpoint: The `/info` endpoint is used to display general information about the application, such as version, description, or custom details like Git commit information.
- Dynamic Configuration: Endpoints like `/env` and `/configprops` are used for managing and reviewing application configuration. In dynamic environments, `/refresh` is used to reload configuration without restarting the application.

5. Security

- Securing Endpoints: Sensitive Actuator endpoints are secured using Spring Security, ensuring that only authenticated users with proper roles can access them.
- HTTPS and Firewall Rules: HTTPS is used for endpoint communication, and firewall rules are often applied to restrict access to Actuator endpoints.

6. Logging and Tracing

- Logging Configuration: Endpoints like `/loggers` are used for viewing and changing logging levels dynamically.
- Tracing: The `/httptrace` endpoint (where enabled) is used for tracing HTTP request-response exchanges, which is helpful for debugging and monitoring.

7. Documentation and Best Practices

- Companies often document their Actuator setup and usage guidelines, ensuring that developers and operation teams are aware of the monitoring and management capabilities available.